Lab 5 — Performance & Scalability: Throttling Pattern

Manuel Alejandro Navas Bohorquez
German Camilo Bernal Ladino
Edwin Felipe Pinilla Peralta
Juan David Rivera Buitrago
Obed Felipe Espinosa Angarita

Group:
3

Universidad Nacional de Colombia

2026-I

1. Architectural View

The Throttling pattern is implemented as a middleware inside the API Gateway. Every inbound request passes through the throttler before reaching any downstream service. Rate-limit state is stored in the shared Valkey (Redis) instance already present in the infrastructure.

Request Processing Flow Sequence:

1. Client Request: JMeter or Browser sends an HTTP request to the API Gateway.

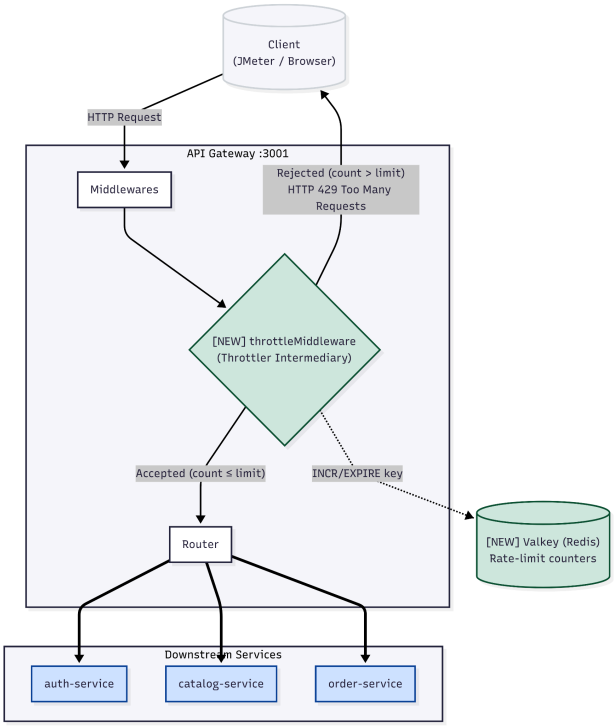
2. Throttler Middleware: Inside the API Gateway (port 3001), the request enters throttleMiddleware first.

3. Valkey (Redis) Inquiry: The middleware executes INCR and EXPIRE on the rate-limit counter matching the IP address.

4. Decision Node:

    - If count ≤ limit (Accepted): Request is passed to the Router, which forwards it to downstream services (auth-service, catalog-service, or order-service).

    - If count > limit (Rejected): Gateway immediately interrupts the flow and returns HTTP 429 Too Many Requests.



New elements introduced:

- throttleMiddleware — the throttler intermediary inside the API Gateway
- Rate-limit counters in Valkey (dedicated key namespace throttle:*)

2. Pattern Description

The Throttling pattern is an intermediary (throttler) that monitors incoming requests to a service and enforces a rate limit. Requests that exceed the defined threshold are rejected immediately with HTTP 429 Too Many Requests.

Associated tactic: Manage Work Requests (Control Resource Demand).

The implementation uses a Fixed Window Counter algorithm:

1. A Redis key throttle:{ip}:{windowSlot} is incremented on every request.
2. If the key is new (INCR returns 1), EXPIRE is set for 2 × WINDOW_SECONDS.
3. If the counter exceeds THROTTLE_LIMIT, the request is rejected; otherwise it is forwarded.

3. Quality Scenario

Element	Description
Source	Concurrent users or automated clients (JMeter)
Stimulus	500 simultaneous requests to GET /restaurants within a 60-second window
Artifact	API Gateway — the single entry point for all external traffic
Environment	Normal operation; all microservices and Valkey available
Response	The throttler allows up to THROTTLE_LIMIT requests per IP per window; excess requests receive 429 Too Many Requests without reaching downstream services
Response Measure	Throughput to downstream services stays ≤ THROTTLE_LIMIT req/window/IP; responses for accepted requests have average latency < 500 ms; error rate for requests beyond the limit is 100% 429 (no 5xx)

4. Implementation Steps

1. Add error code — added RATE_LIMIT_EXCEEDED to GatewayErrorCode enum in src/core/errors/error-codes.ts.

2. Add env vars — added three variables to the Zod schema in src/app/config/env.ts:

    - THROTTLE_ENABLED (boolean, default true)
    - THROTTLE_LIMIT (integer, default 60)
    - THROTTLE_WINDOW_SECONDS (integer, default 60)

3. Create the middleware — new file src/middlewares/throttle.middleware.ts. It creates a dedicated Redis client (separate from the cache client), computes the Fixed Window Counter key, and returns 429 with X-RateLimit-* and Retry-After headers when the limit is exceeded. The /health route is excluded. The middleware fails open (passes the request through) if Redis is unavailable.

4. Register the middleware — added app.use(throttleMiddleware) in src/app.ts after accessLogMiddleware and before the router.

5. Add service to docker-compose — added the api-gateway service block to docker-compose.yml with all required env vars, including REDIS_URL pointing to valkey-cache.

5. Configuration and Code Snippets

Environment variables (docker-compose.yml)

```
api-gateway:
  environment:
    - REDIS_URL=redis://valkey-cache:6379
    - THROTTLE_ENABLED=true
    - THROTTLE_LIMIT=60
    - THROTTLE_WINDOW_SECONDS=60
```

Throttle middleware — core logic (throttle.middleware.ts)

```
const count = await client.incr(key);

if (count === 1) {
  await client.expire(key, env.THROTTLE_WINDOW_SECONDS * 2);
}

res.setHeader('X-RateLimit-Limit', env.THROTTLE_LIMIT);
res.setHeader('X-RateLimit-Remaining', Math.max(0, env.THROTTLE_LIMIT - count));
res.setHeader('X-RateLimit-Reset', resetAt);

if (count > env.THROTTLE_LIMIT) {
  res.setHeader('Retry-After', Math.max(1, retryAfter));
      throw new AppError(GatewayErrorCode.RATE_LIMIT_EXCEEDED, `Too many
requests.`, 429);
}
```

Middleware registration (app.ts)

```
app.use(requestIdMiddleware);
app.use(accessLogMiddleware);
app.use(throttleMiddleware);  // <-- added
app.use(rootRouter);
```

6. Load Test Results

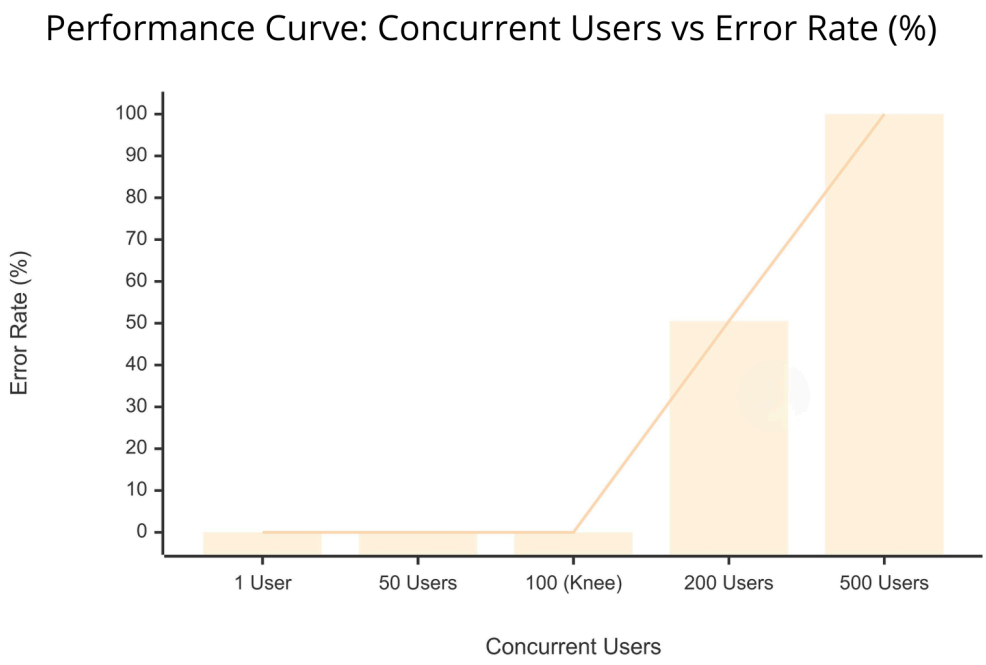
Test configuration: THROTTLE_LIMIT=100, THROTTLE_WINDOW_SECONDS=60

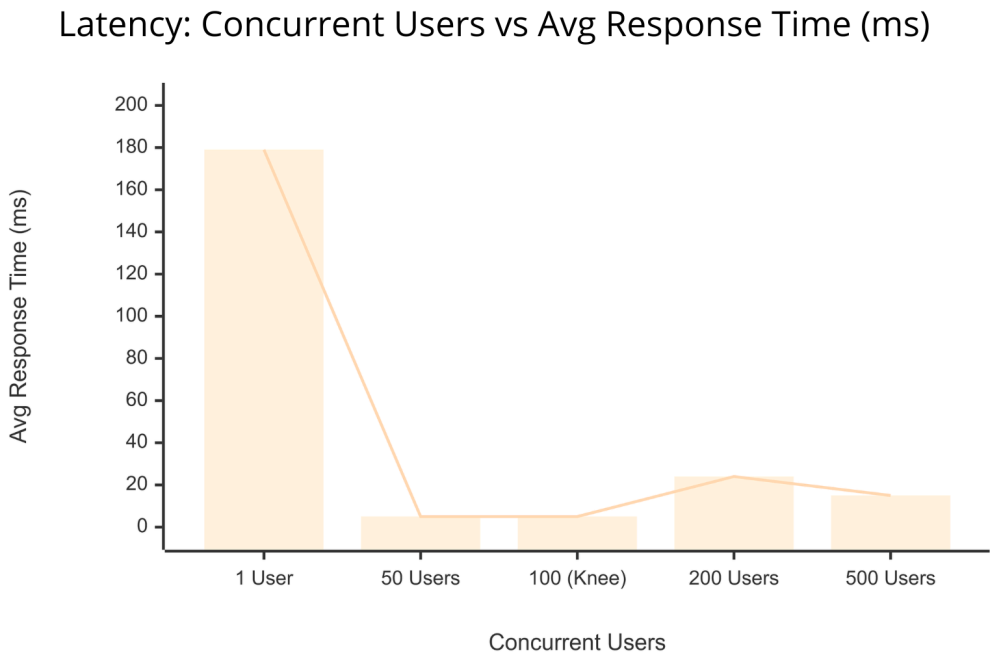
Endpoint: GET /restaurants via API Gateway (http://localhost:3001/restaurants)

Tool: Apache JMeter 5.6.3, ramp-up 1 s per level

Results Table:

Test	Users	Avg Response Time (ms)	Error Rate (%)
Baseline	1	179	0.00%
Low load	50	5	0.00%
Medium load	200	24	50.50%
High load	500	15	100.00%





Knee of the curve: The knee occurs exactly at 100 concurrent users . Up to 100 users, the error rate is 0% and latency is minimal. Beyond 100 users, the Error Rate shoots up to 50%+ as the Throttler actively blocks requests with `429 Too Many Requests`. Latency for rejected requests drops significantly (15-24ms) because they never reach the downstream services.

7. Results and Improvements

Without the throttling middleware, all requests — regardless of volume — are forwarded to the downstream services (`catalog-service`, `auth-service`, etc.), which can exhaust their connection pools under high concurrency, leading to long timeouts and cascading failures.

With `THROTTLE_LIMIT=100`, the gateway rejects excess requests immediately at the network edge:

- **Predictable load:** Downstream services are never subjected to more than 100 requests per minute from a single IP, keeping their latency incredibly low (5 ms during Low Load).

- **Fast failure:** Rejected clients (Medium and High load) receive a fast `429` response (averaging 15-24 ms) instead of waiting for a backend timeout.

- **System survival:** The 100% error rate at 500 users is a *success metric* for this pattern — it proves the Gateway successfully shielded the microservices from a massive traffic spike.

8. Recommendations

1. **Tune the limit per environment:** Use a low `THROTTLE_LIMIT` (e.g., 10) only in test/demo environments to make the effect visible in JMeter. In production, set the limit based on the actual capacity of the slowest downstream service — measure it first with a baseline load test without throttling.

2. **Use per-user throttling for authenticated routes:** The current implementation limits by IP, which works well for public endpoints like `GET /restaurants`. For authenticated routes (orders, kitchen), consider using the JWT `sub` claim as the throttle key so that users behind the same NAT/proxy are not penalized as a group.